Congame Bot Testing: Gap Analysis (v3)

Bots vs Playwright — corrected and sharpened

conmage

2026-05-10

Architecture

Both congame bots and Playwright tests drive real browsers. Bots use Marionette/Firefox; Playwright uses Chromium. Both can read rendered pages, interact with DOM elements, and fill forms.

Bots additionally have `page-execute-async!` which runs arbitrary JavaScript in the browser context — meaning they can read any page content, check text, and verify rendered output.

Current Usage Gap

The gap is not architectural but conventional. Existing bot models use only:

- `bot:continuer` — click next
- `bot:autofill` — fill form presets
- `bot:find` — check element existence (for wait pages)
- `bot:completer` — end study

No existing bot model reads page text or asserts content correctness. This is a skill/convention problem, not a framework limitation.

Claimed Gaps — Reassessed

"Design fidelity" — Not a Playwright advantage

The previous report claimed Playwright tests verify design fidelity while bots can't. This is wrong. Design-fidelity testing comes from **Claude reading the design document and writing assertions** — the test framework is just the execution engine. The same process works for bot models: Claude reads the design doc, writes a bot model with content assertions using `page-execute-async!` to read page text and verify expected content.

"Structured narratives" — Not a Playwright advantage

The previous report noted Playwright tests have narrative headers explaining the design. This is a code comment convention, not a framework feature. Bot models can (and should) have the same narrative comments.

Concurrency — Real difference, partially bridgeable

Playwright runs multiple participants concurrently with shared test scope (`Promise.all` + shared variables). Bots run in separate threads with no shared scope.

However, bots have an advantage Playwright doesn't: they can read server-side shared state. A bot could read `defvar/instance` values (or their effects on rendered pages) to verify cross-participant correctness. For example, after a matchmaking wait resolves, a bot could verify the opponent's score appears correctly by reading the rendered page.

The remaining Playwright edge: coordinating assertions *across* participants in a single test (e.g., "participant A reported 70, so participant B's results page should show 70"). Bots would need a test harness collecting results from independent bot runs to do this.

Playwright-only capabilities — None identified

Since bots drive a real browser and can execute arbitrary JS, there is no category of test that Playwright can run but bots fundamentally cannot. The differences are in ergonomics and conventions, not capability.

Bot-only capabilities

Bots can access server-side state that never reaches the browser:

- `defvar/instance` values (shared state) — visible via rendered effects but also potentially readable directly
- Server-side logs (`log-conscript-info`)
- Transaction behavior (retry logic in `with-study-transaction`)

Playwright can only see what the browser sees. If a variable is set but never rendered, Playwright can't verify it.

Revised Comparison

Capability	Bots (current)	Bots (with richer models)	Playwright
Page content assertions	Not used	Yes (`page-execute-async!`)	Yes
Design-fidelity testing	Not used	Yes (Claude writes from design doc)	Yes

Capability	Bots (current)	Bots (with richer models)	Playwright
Narrative documentation	Not used	Yes (code comments)	Yes
Cross-participant assertions	No	Partial (via rendered effects)	Yes (shared scope)
Server-side state access	Not used	**Yes (bot-only advantage)**	No
Setup cost	Low (Racket only)	Low	Higher (Node, Docker)

TODO: How to Extend Bots

1. **Add page text reading to bot models.** Use `page-execute-async!` with `document.querySelector('div.container').textContent` to read page content. Assert expected strings.
2. **Generate richer bot models in create-study.** When Claude writes a bot model from a design doc, include content assertions: verify payment amounts, treatment-specific text presence/absence, instruction correctness.
3. **Add narrative comments to bot models.** Same convention as Playwright test headers — explain what the design specifies and what each assertion checks.
4. **Add more bot kinds.** Generate 3–4 kinds covering edge cases (zero input, max input, tie scenarios), not just 2.
5. **Consider a bot test harness for cross-participant assertions.** Collect results from N independent bot runs into a shared summary, then assert cross-participant properties. Lower priority since this is the one area where Playwright is genuinely more ergonomic.
6. **Keep Playwright for integration testing.** Its advantage is ergonomic (shared scope, familiar JS syntax, easy CI integration), not fundamental. Use bots for fast per-study testing, Playwright for full integration suites.